

Expenses Reporting Tool for an IT Consultancy

Project Management (GEP)
Assignment 4: Final Document

Bachelor Degree in Informatics Engineering
Specialization in Software Engineering

Bachelor's Thesis



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona



Natalia Yike Wang Yang

Thesis Supervisor

Sergio Alejandro Lobig Machado (FSP Consulting Services Limited)

Thesis Director

Marc Oriol Hilari (Department of Software Engineering)

GEP Tutor

Eduard Balbuena Longo (Department of Project Management)

Universitat Politècnica de Catalunya
Facultat Informàtica de Barcelona
Computer Science Department

March 11, 2026

Contents

5	Temporal Planning	1
5.1	Task Description	1
5.1.1	Project Starting	1
5.1.2	Project Management	1
5.1.3	Final Project Delivery	2
5.1.4	Design	2
5.1.5	Development	3
5.1.6	Frontend and Backend Development	3
5.1.7	Backend Development	5
5.1.8	Frontend Development	6
5.2	Resources	6
5.2.1	Human Resources	6
5.2.2	Physical Resources	6
5.3	Summary of Tasks and Resources	7
5.4	Gantt Diagram	10
5.5	Risk Management	11

List of Figures

5.4.1 Gantt Diagram (own creation)	11
--	----

List of Tables

5.2.1 Human Resources.	6
5.2.2 Hardware Resources.	6
5.2.3 Software Resources.	7
5.3.1 Summary of project tasks with effort, dependencies, and resources	9
5.5.1 Summary of identified risks, their probabilities, affected tasks, and contingency resources.	13

5 Temporal Planning

This section introduces the initial **122-day** project plan, scheduled from February 18, 2026, to June 19, 2026. The total estimated effort for all defined tasks is **513 hours**.

According to Polytechnic University of Catalonia, the bachelor's thesis carries a workload of 18 ECTS credits, equivalent to 540 effort hours. The discrepancy between the total estimated effort of 513 hours and the expected 540 hours results in a built-in buffer of **27 hours** to accommodate potential setbacks.

5.1 Task Description

The key to achieving effective time planning lies in a thorough analysis of all tasks, their priorities and durations. This step facilitates proper task sequencing and resource allocation. For this reason, all tasks are outlined in this section and categorized into six main groups: project starting, project management, final project delivery, design, development, and frontend/backend development.

5.1.1 Project Starting

A solid project start requires a preliminary study and initial project definition. This aligns the project vision among all involved members and establishes a unified understanding of the project's scope and objectives. To achieve this, the subsequent tasks are defined:

PS1. Context and Scope

This task is divided into smaller sub-tasks:

- Analyze the current problem and identify all stakeholders.
- Define and propose a technical solution.
- Describe the project context and conduct a market study.
- Define the project scope in terms of objectives, requirements, obstacles, and risks.
- Select a pattern methodology to guide the entire project development.
- Select the monitoring and validation tools to be used in the project.

PS2. Temporal Planning

Identify all tasks and resources, design an initial Gantt diagram timeline, and define a risk management strategy.

PS3. Budget and Sustainability

Define a project budget and analyze the sustainability aspects of the proposed solution.

PS4. Initial Documentation

Consolidate the documentation from previous tasks into the definitive version of the initial documentation.

5.1.2 Project Management

Project management is essential for monitoring the project's progress and keeping the project on track.

PM1. Agile Ceremonies

Run daily scrums, sprint plannings, sprint reviews, and sprint retrospectives.

PM2. Thesis Director Review Meetings

Meet with the thesis director every two weeks to ensure the project is progressing correctly.

PM3. Product Backlog Update

Refine, update, and maintain the product backlog, which always reflects the current project status.

5.1.3 Final Project Delivery

Tasks for completing product delivery are listed below.

FPD1. User Manual Documentation

Write a user manual for the application's end-users as a reference guide.

FPD2. Final Thesis Documentation

Write the bachelor's thesis final documentation intended for an academic audience. This task involves structuring the full project development, from planning and design to results and conclusions, maintaining an academic tone and standard.

FPD3. Final Demonstration Video

Record a final demonstration of the application once it is ready for use.

FPD4. Bachelor's Thesis Defense

Plan and prepare a final exposition to defend and justify the bachelor's thesis achievements, while demonstrating commitment to the academic standards.

5.1.4 Design

This preliminary design phase establishes a clear application architecture to guide the subsequent development phase.

DT1. Use Case Diagram

Analyze all users' interactions with the system and design a Use Case Diagram that illustrates all actors, use cases, and relationships between use cases.

DT2. Conceptual Model Diagram

Design the UML database diagram, clarifying all database entities, their attributes, and the relationships between them.

DT3. Technology Stack Decision

Select the technology to be used for each application layer and provide a clear technology selection documentation that serves as a guide for the application development phase.

DT4. Logical and Physical Architecture Design

Design the logical and physical application architecture.

The former describes how the implementation code is structured, including the purpose and role of each application layer. The latter explains how the application connects to external services and resources, both physical and virtual, and describes the function of each.

DT5. UI mockups Design

Deliver the mockup designs for each application page.

DT6. Security Design

Provide a security architecture design for user authentication, information submission and storage, and access control.

5.1.5 Development

Essential tasks for the application development are specified in this task group.

Dev0. Self-learning and study of unfamiliar technologies

Study and learn new application technologies in advance.

Dev1. Environment Configuration

Configure a ready-to-use environment to begin application development. This task involves installing Visual Studio Code, essential extensions, creating a Git repository, and similar setup tasks.

Dev2. Product Deployment

Release the final product for end-user availability.

Dev3. Unit and Integration Testing

Include testing to verify that the implemented code functions correctly and to prevent coding bugs.

Dev4. Logging and Monitoring

Set up logging and monitoring tools to track application performance and review logs.

Dev5. Git Repository Documentation

Maintain clear repository documentation, including an application setup guide, API contracts documentation, and meaningful code comments. This documentation is targeted to a technical audience.

5.1.6 Frontend and Backend Development

As a full-stack web application, the implementation is divided into two main areas. The frontend is responsible for the user interface (UI) and user experience (UX). The backend implements the complex implementations hidden in the frontend via a RESTful API and integrates external services.

This development stage requires both frontend and backend development. Since it covers a wide range of functionalities, it is divided into two subgroups: Expense and Report Development, and User Access Development.

Expense and Report Development

This section includes all tasks related to the application's core functionalities, namely the implementation of the expense and report features.

ER1. Expense Uploads

Develop a file submission feature where the frontend handles the expense drag-and-drop and upload interface, and the backend registers the uploaded expense in the database.

ER2. Uploaded Expenses Real-time Dashboard

Develop a visual dashboard to display all uploaded expenses with their real-time processing status, which can be:

- **Processing:** The expense is pending processing by the AI model.
- **Pending Review:** The AI model has successfully analyzed the expense, the uploader can now review it.
- **Completed:** The user has reviewed and completed all required information. The expense is ready for being included in a new or existing report.
- **Failed:** The AI couldn't extract data from the receipt (due to unavailability or unrecognized format).

ER3. AI-Extracted Expense Data Review and Correction

Develop a form page for users to review and edit AI-processed expenses and fill in missing required information. Once the expense information is saved, the database is updated with the corrected data.

ER4. Uploaded Expenses Deletion

Develop the expense deletion feature to delete one or multiple expenses at once, regardless of their current status. When the user confirms deletion, the respective expenses are marked as deleted in the database.

ER5. Expense Reports View, Creation, Edit, Submission, and Deletion

Develop report management functionalities (create, edit, submit, and delete reports) and the required RESTful API for synchronizing the database with the frontend changes.

User Access Development

The tasks described in this subgroup are contingent upon user authentication implementation before their execution can begin. It is important to note that administrators can set spending limits and manage and view submitted reports solely from employees under their supervision.

UA1. Company Email User Authentication

Develop the login feature that allows users to authenticate in the application through the company's email account:

1. Verify the user access.
2. (a) On the first login, create a new user account.
(b) On the return visit, recover the relevant data of the corresponding user.

UA2. Role-based User Access

Develop the user role assignment (based on two main roles: regular employee or administrator) and expose only the corresponding role's functionalities.

UA3. Submitted Report Approval and Rejection

Develop a feature to allow administrators to review, approve, or reject submitted reports, and update the database accordingly.

UA4. Employee Spending Limit Configuration

Develop an admin-only interface to set employees' spending limits and record the changes in the database.

UA5. Submitted and Reviewed Expenses Records Visualization

Develop a dashboard displaying the records of both users' submitted reports and administrators' reviewed reports.

UA6. Email Notification Service Integration

Develop the email notification service to notify users when their submitted reports are reviewed, and to administrators when new reports are pending for their review.

5.1.7 Backend Development

The following tasks are implemented exclusively in the application backend logic.

BD1. RESTful API Backend Logic Implementation

Configure the backend architecture defined in the DT3 task and build a RESTful API to manage backend responsibilities through frontend requests.

BD2. Azure SQL Database Integration

Integrate Azure SQL Database with Entity Framework Core migrations to manage users' relevant information (e.g., extracted expense and report information, employees' hierarchy) and configure the initial database according to the diagram designed in DT2 task.

BD3. Azure Blob Storage Integration

Store original expense files using Azure Blob Storage.

BD4. Microsoft Document Intelligence AI Model Integration

Automate the expense data extraction through the Microsoft Document Intelligence AI model integration.

BD5. Automated AI-driven Data Extraction Pipeline

Integrate Event Grid, Service Bus, and Azure Function and implement a mechanized data extraction workflow that prevents bottlenecks during multiple simultaneous uploads:

1. Save the uploaded receipt file in the Blob Storage and store the receipt information in the SQL Database.
2. Detect the event using Azure Event Grid and route the expense file to the Service Bus queue.

3. Once the new expense is in the queue, trigger the Azure Function to process the receipt file using the integrated AI model.

BD6. Azure SignalR Real-time Notification Service Integration

Integrate SignalR to configure a SignalR Hub that sends real-time expense status updates, eliminating manual page refreshing whenever the AI processes a new receipt file.

BD7. Microsoft Entra ID User Authentication Integration

Handle employee authentication through the company email account by using Microsoft Entra ID integration.

5.1.8 Frontend Development

This section covers all tasks that require only frontend logic implementation.

FD1. Expenses Search and Filter

Develop visual tools that enable users to search uploaded expenses by name or filter them by category.

FD2. Reports Search

Provide a search bar to search specific reports by name.

5.2 Resources

Following the analysis of all required tasks for successful project completion, all necessary resources must be identified. Resources are divided into two main groups: human resources and physical resources. All human resources are listed in Table 5.2.1, specifying the responsible individual for each role. The physical resources are further categorized into software and hardware resources, and are identified and briefly described in Tables 5.2.2 and 5.2.3.

5.2.1 Human Resources

Resource ID	Role Name	Responsible Individual
SM	Scrum Master	Thesis author
PO	Product Owner	Sergio Alejandro Lobig Machado
TD	Thesis Director	Marc Oriol Hilari
GT	GEP Tutor	Eduard Balbuena Longo
DT	Development Team	Thesis author

Table 5.2.1: Human Resources. [Own creation]

5.2.2 Physical Resources

Hardware Resources

Resource ID	Resource Name	Components
OS	Office Supplies	A table, a chair, electricity, internet, and office rental.
HW	Hardware Resources	A computer, two monitors, a mouse, and a keyboard.

Table 5.2.2: Hardware Resources. [Own creation]

Software Resources

Resource		
ID	Resource Name	Description
GIT	Git	A version control tool for managing the source code repository.
AZR	Microsoft Azure Resources	Cloud services and resources: SQL Database, Blob Storage, AI model, Event Grid, Service Bus, SignalR, EntraID, and Azure DevOps (for repository and product backlog management).
MCS	Microsoft Collaboration Tools	Communication and coordination platforms: Microsoft Teams, Calendar, Outlook, and Word.
VSC	Visual Studio Code	Source code editor with extensions and debugging tools.
FIB	FIB Platforms	Platforms from Barcelona School of Informatics(FIB): Racó, Atenea, FIB secretary, bachelor's thesis official website guide.
GM	Gmail	Email communication with university entities.
WEB	Web Browsers and Search Engines	Research Browsers: Mozilla Firefox, Google Chrome, Microsoft Edge.
OVL	Overleaf LaTeX Editor	Thesis formatting editor for final thesis documentation.
OT	Online Tools	Diagram design tools: Edraw, Draw.io, and Gantt Project.

Table 5.2.3: Software Resources. [Own creation]

5.3 Summary of Tasks and Resources

See Table 5.3.1 for a summary of all defined tasks, including their identifiers, estimated effort in hours, any final-to-start dependencies with other tasks, and the required resources. The final-to-start dependency column indicates which task must be fully completed before the task in that row can begin.

ID	Name	Time (h)	Dependencies	Resources
PS	Project Starting	50		
PS1	Context and Scope	20	-	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS2	Temporal Planning	10	PS1	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL, OT
PS3	Budget and Sustainability	10	PS2	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS4	Initial Documentation	10	PS3	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PM	Project Management	67		
PM1	Agile Ceremonies	51	PS2	SM, PO, DT, OS, HW, AZR, MCS, VSC
PM2	Thesis Director Review Meetings	8	-	DT, TD, OS, HW, AZR, VSC, FIB, GM, WEB
PM3	Product Backlog Update	8	PS2	SM, DT, PO, OS, HW, AZR, WEB
FD	Final Product Delivery	46		
FPD1	User Manual Documentation	2	PM5	DT, OS, HW, VSC, WEB
FPD2	Final Thesis Documentation	20	-	DT, TD, OS, HW, FIB, GM, WEB, OVL, OT
FPD3	Final Demonstration Video	4	FPD2	DT, OS, HW, WEB
FPD4	Bachelor's Thesis Defense	20	FPD2	DT, TD, OS, HW, FIB, GM, WEB
DT	Design	16		
DT1	Use Case Diagram	2	PS1	DT, OS, HW, OT, WEB, MCS
DT2	Conceptual Model Diagram	4	PS1	DT, OS, HW, OT, WEB
DT3	Technology Stack Decision	2	PS1	DT, OS, HW, WEB, MCS
DT4	Logical and Physical Architecture Design	4	PS1	DT, OS, HW, WEB, MCS
DT5	UI mockups Design	2	DT1	DT, OS, HW, OT, WEB
DT6	Security Design	2	DT3, DT4	DT, OS, HW, WEB, MCS
Dev	Development	76		
Dev0	Self-learning and study of unfamiliar technologies	8	DT3	DT, OS, HW, WEB
Dev1	Environment Configuration	2	DT2, DT5, DT6	DT, OS, HW, GIT, AZR, VSC, WEB
Dev2	Product Deployment	8	Dev3	DT, OS, HW, GIT, AZR, VSC, WEB
Dev3	Unit and Integration Testing	10	UA4, UA5, UA6	DT, OS, HW, GIT, AZR, VSC, WEB
Dev4	Logging and Monitoring	8	Dev2	DT, OS, HW, GIT, AZR, VSC, WEB
Dev5	Git Repository Documentation	40	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
Frontend and Backend Development		128		
ER	Expense and Report Development	64		
ER1	Expense Uploads	16	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
ER2	Uploaded Expenses Real-time Dashboard	4	ER1, BD6	DT, OS, HW, GIT, AZR, VSC, WEB
ER3	AI-Extracted Expense Data Review and Correction	20	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
ER4	Uploaded Expenses Deletion	4	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	ER3, ER4	DT, OS, HW, GIT, AZR, VSC, WEB
UA	User Access Development	64		

(Continued on next page)

(Continued from previous page)

ID	Name	Time (h)	Dependencies	Resources
UA1	Company Email User Authentication	8	BD7	DT, OS, HW, GIT, AZR, VSC, WEB
UA2	Role-based User Access	16	UA1	DT, OS, HW, GIT, AZR, VSC, WEB
UA3	Submitted Report Approval and Rejection	8	ER5, UA2	DT, OS, HW, GIT, AZR, VSC, WEB
UA4	Employee Spending Limit Configuration	8	UA2	DT, OS, HW, GIT, AZR, VSC, WEB
UA5	Submitted and Reviewed Expenses Record Visualization	4	UA3	DT, OS, HW, GIT, AZR, VSC, WEB
UA6	Email Notification Service Integration	20	UA3	DT, OS, HW, GIT, AZR, VSC, WEB
BD	Backend Development	120		
BD1	RESTful API Backend Logic Implementation	20	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
BD2	Azure SQL Database Integration and Manipulation	20	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
BD3	Azure Blob Storage Integration	8	BD2	DT, OS, HW, GIT, AZR, VSC, WEB
BD4	Microsoft Document Intelligence AI Model Integration	20	BD2, BD3	DT, OS, HW, GIT, AZR, VSC, WEB
BD5	Automated AI-driven Data Extraction Pipeline	20	BD4	DT, OS, HW, GIT, AZR, VSC, WEB
BD6	Azure SignalR Real-time Notification Service Integration	16	BD5	DT, OS, HW, GIT, AZR, VSC, WEB
BD7	Microsoft Entra ID User Authentication Integration	16	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
FD	Frontend Development	10		
FD1	Expenses Search and Filter	8	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
FD2	Reports Search	2	ER5	DT, OS, HW, GIT, AZR, VSC, WEB
Total		513		

Table 5.3.1: Summary of project tasks with effort, dependencies, and resources. [Own creation]

5.4 Gantt Diagram

A Gantt diagram is attached at the end of this section. Continuous arrows represent Finish-Start (FS) dependencies between tasks, while dashed arrows represent Finish-Finish (FF) dependencies. FS dependencies indicate that successor tasks cannot start until their predecessor tasks are completed. FF dependencies show that successor tasks cannot finish until their predecessor tasks are completed.

Referring to Diagram 5.4.1, the project timeline spans 122 days. Considering the 513 total estimated hours, this equates to approximately **4 hours of work per day**. Consequently, the weekly workload is 28 hours, matching the weekly effort allocated in the Gantt diagram. Hence, the task effort estimations from Table 5.3.1 are translated into the Gantt diagram using this pattern (e.g., the **ER1** task, which costs 20 hours, represents 4 days of work in the diagram).

Since the Agile Methodology is selected for this application development, each week in the Gantt diagram represents one sprint. To perform the Agile ceremonies described in Section 4.1, daily Scrum (15 minutes each) is scheduled every 2 days, resulting in 3 meetings per week that happen on every Monday, Wednesday, and Friday. Sprint planning, review, and retrospective require 45 minutes each. Thus, Agile ceremonies total 3 hours per week, which equals 51 hours over 17 weeks. Agile's iterative meeting structure spans the **PM1** task across the entire Gantt diagram timeline. However, this task **should be represented iteratively**, since the Gantt diagram tool does not support this behaviour, it is represented as a concurrent task.

Other project management tasks follow the same pattern:

- **PM2** remains undetermined due to undefined thesis director meetings, representing iterative appointments throughout the project lifetime.
- **PM3** starts after **PS2** (temporal planning) is completed and runs concurrently with full application development, as the product backlog requires constant updates whenever new features or tasks are added.
- **FPD2** runs over the project stage to ensure consistency with final deliverables.
- **Dev5**, which depends on **Dev1** (beginning once the repository is created), spans most of the project duration to satisfy objective SG7 from Section 3.1, as clean code requires ongoing, extensive documentation.

Certain development tasks also demand additional time. These tasks have an initial effort estimation of 8 hours (2 days of the Gantt diagram). Since these tasks face a high probability of experiencing significant challenges, the remaining effort time is allocated as a buffer.

For example, **ER1**, **ER3**, **UA2**, and **UA6**. **ER1**. **ER1** is the first coding task; **ER3** requires extensive coordination with frontend and backend; **UA2** requires a prior database record of the employee hierarchy; **UA6** marks the author's first experience implementing email notifications. Finally, **Backend Development** tasks similarly incur additional time costs.

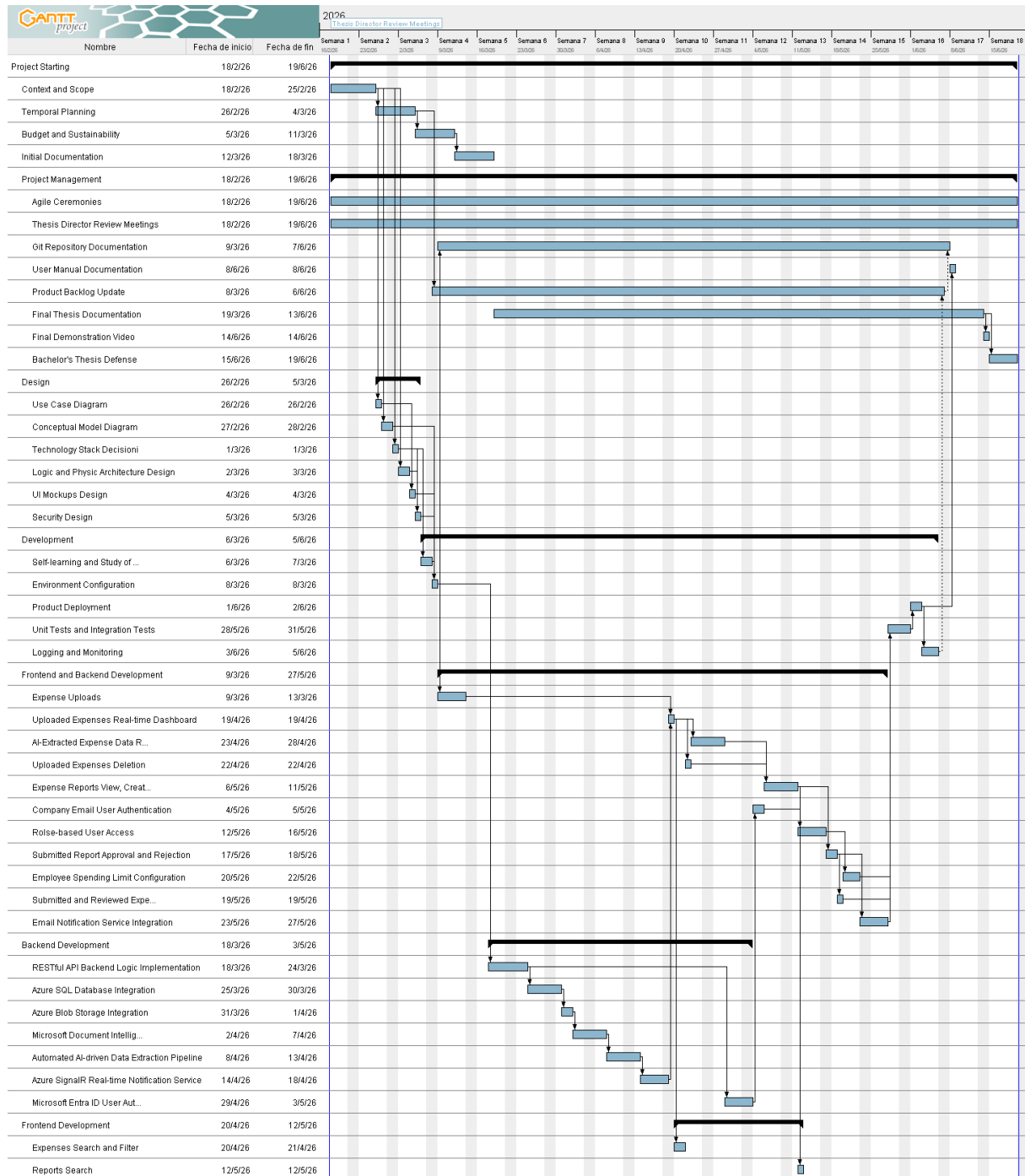


Figure 5.4.1: Gantt Diagram (own creation)

5.5 Risk Management

Possible risks affecting project progress are outlined in Section 3.3. With the timeline now established, contingency plans are defined to mitigate potential obstacles. This section presents each identified risk along with its probability, impact, and corresponding mitigation strategy. Moreover, all identified risks, along with the corresponding affected tasks and the contingency resources required for mitigation, are detailed in Table 5.5.1.

Coding Bugs

Probability: Medium

Impact: High

Coding bugs are inevitable, making thorough testing essential. As outlined in Section 5.4, every code change undergoes rigorous testing across all use cases, including edge cases. During the Dev1 task, ESLint and SonarQube Visual Studio Code extensions are installed to automate bug detection from the very beginning of the implementation.

If bugs persist, additional debugging time is allocated, using **10 hours** of the buffer time. It is important to fix critical bugs before proceeding with new feature development, prioritizing functional application delivery.

However, thanks to the installed extensions and consistent testing, the number of bugs is minimized, reducing the likelihood of significant setbacks. Furthermore, complex tasks already include built-in buffer time, as mentioned in section 5.4, and the reserve hours are available to cover any delays that may arise.

Tight Project Deadlines**Probability: High****Impact: Medium**

Software development projects often struggle with tight deadlines. Due to the author's limited professional experience, deviation between the actual task effort and the initial time estimation is anticipated. However, since this project follows Agile methodology, iterative meetings help to detect and manage overruns at an early stage.

To overcome this challenge, the remaining **9 hours** of overhead serve as a lifesaver. This is an intentional allocation, as the project is scheduled to finish several days before the thesis defense period. Therefore, a one-week delay is permitted without affecting the final submission.

In the worst-case scenario, if the project misses the expected deadline, certain non-essential application features will be deferred to the final phase, depending on the time available. Nevertheless, this alternative can only be made after obtaining the Product Owner's approval.

Lack of Experience in Certain Technologies**Probability: High****Impact: Medium**

Given that several technologies used in application implementation lie beyond the author's expertise, mitigation plans have been established to address this challenge. Task Dev0 specifically accounts for this difficulty by allocating dedicated time for self-study, thereby minimizing potential setbacks.

Since a prior study cannot fully mitigate this risk, a proposed solution is to set aside up to **8 hours** from the available buffer. If delays are prolonged, assistance from the FSP team serves as a fallback. Seeking timely assistance is an essential professional skill, as it helps prevent unnecessary deadline extensions.

Company's licenses and security permissions**Probability: Medium****Impact: Low**

Another risk involves FSP's security system. Cyberattacks are taken more seriously every

day, often requiring developers to request and wait for permission to install resources and extensions. Consequently, new restrictions may limit initial resource availability.

To prevent this blocker, Dev1 has been created to install all the previously identified tools and extensions in advance. However, this prior setup does not remove the probability of installing forthcoming tools.

As a contingency, a reordering of the project backlog can be considered using the Gantt chart (Figure 5.4.1) as a reference. Tasks with no dependencies, such as BD7 and the subsequent UA category tasks, may be prioritized and initialized earlier. Likewise, other non-dependent tasks (e.g., Dev5 task) may also be advanced to maintain project progress.

Risk	Probability	Time (h)	Affected Tasks	Resources
Coding Bugs	15%	10	Dev3, ER1-5, UA1-6, BD1-7, FD1-2	DT, HW
Tight Project Deadlines	30%	9	Dev2-4, ER1-5, UA1-6, BD1-7, FD1-2	DT, HW
Lack of Experience in Certain Technologies	30%	8	Dev2, Dev3, ER1-5, UA1-6, BD1-7,	DT, HW
Company's licenses and security permissions	15%	0	Dev2, BD1-7	DT, HW

Table 5.5.1: Summary of identified risks, their probabilities, affected tasks, and contingency resources..
[Own creation]